

A Configurable Multi-Precision CNN Computing Framework Based on Single Bit RRAM

Zhenhua Zhu^{*†}, Hanbo Sun^{*†}, Yujun Lin[‡], Guohao Dai[‡], Lixue Xia[§]
Song Han[‡], Yu Wang[‡], Huazhong Yang[‡]

[†]Dept. of EE, BNRist, Tsinghua University, [‡]Dept. of EECS, Massachusetts Institute of Technology, [§]Alibaba Group
yu-wang@tsinghua.edu.cn

ABSTRACT

Convolutional Neural Networks (CNNs) play a vital role in machine learning. Emerging resistive random-access memories (RRAMs) and RRAM-based Processing-In-Memory architectures have demonstrated great potentials in boosting both the performance and energy efficiency of CNNs. However, restricted by the immature process technology, it is hard to implement and fabricate a CNN accelerator chip based on multi-bit RRAM devices. In addition, existing single bit RRAM based CNN accelerators only focus on binary or ternary CNNs which have more than 10% accuracy loss compared with full precision CNNs. This paper proposes a configurable multi-precision CNN computing framework based on single bit RRAM, which consists of an RRAM computing overhead aware network quantization algorithm and a configurable multi-precision CNN computing architecture based on single bit RRAM. The proposed method can achieve equivalent accuracy as full precision CNN but also with lower storage consumption and latency via multiple precision quantization. The designed architecture supports for accelerating the multi-precision CNNs even with various precision among different layers. Experiment results show that the proposed framework can reduce 70% computing area and 75% computing energy on average, with nearly no accuracy loss. And the equivalent energy efficiency is 1.6 ~ 8.6 $\times$ compared with existing RRAM based architectures with only 1.07% area overhead.

1 INTRODUCTION

Nowadays, Convolutional Neural Networks (CNNs) prevail and make breakthroughs in many fields. However, CNN models are becoming more and more complex, with the amount of parameters and calculations become huger. Thus, both the training and inference of state-of-the-art CNNs cause high energy consumption and long computation time.

Previous work demonstrated that emerging resistive random access memories (RRAMs) provide alternative solution to boost the energy efficiency of CNN computing [3] due to its capability of performing matrix-vector-multiplications (MVMs) in memory, referred as one of the Processing-In-Memory (PIM) architectures. Existing work has demonstrated that RRAM-based PIM architectures can enhance the energy efficiency by over 100 $\times$ compared with both

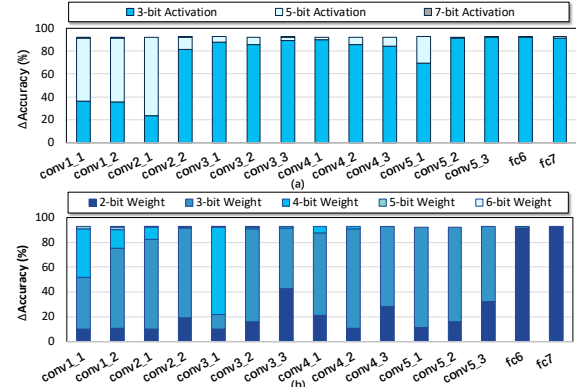


Figure 1: The accuracy increment of VGG as the precision of (a) activations or (b) weights increases

CMOS ASIC and GPU solutions [3, 15, 16]. In addition, some RRAM-based accelerators utilize the multi-bit RRAM devices which have more than two stable resistance states, to achieve higher storage and computation density, such as PRIME [3] and PipeLayer [16] with 4-bit RRAM. Despite of the high density of multi-bit RRAM, there exist three major problems of the multi-bit RRAM based architectures, making them hard to realize.

First, it is difficult to implement and tape out a multi-bit RRAM based CNN accelerator due to the immature manufacturing process. Up to now, there is few fabricated multi-bit RRAM crossbar for computing. Hu et al. [6] fabricated a 4×4 crossbar based on 5-bit RRAM cells, but it is far from enough for CNN with typically over one million parameters. Second, the existing multi-bit devices are not suitable for computation due to non-ideal factors. The resistance non-linear problem of multi-bit RRAM degrades the accuracy severely, and has not been addressed well [12]. Meanwhile, compared with the single bit RRAM, the multi-bit RRAM suffers more from resistance variations, which harm the CNNs performance greatly [18]. Third, the quantization errors from the analog and digital interfaces of multi-bit RRAM crossbars are non-negligible. The precision of the interfaces is limited by the area and power constraints, noticeable hardware quantization errors are introduced causing accuracy degradation.

In order to achieve high reliable and accurate CNN computing, researchers and chip designers also proposed several CNN accelerators based on single bit RRAM [2, 17, 18, 21]. These accelerators mainly accelerate binary CNNs (BCNN) where a binary/ternary weight can be expressed by one/two single bit RRAM. However, on account of the limited weight precision, *i.e.*, binary, the accuracy loss of BCNN is around 10 ~ 20% on ImageNet dataset [13, 23].

Besides, Figure 1 implies that the precision requirements of each layer are different. It means that the layer-wise quantization, *i.e.*,

*: Both authors contributed equally to this work.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

DAC '19, June 2–6, 2019, Las Vegas, NV, USA

© 2019 Association for Computing Machinery.

ACM ISBN 978-1-4503-6725-7/19/06...\$15.00

<https://doi.org/10.1145/3316781.3317739>

Table 1: The Performance of the Different Quantization Results on GPU and RRAM (VGG on CIFAR-10, with 20% weights precision and 50% activation precision)

	Baseline	GPU Quant.	GPU Quant. On RRAM	Ours
Accuracy	92.02%	88.80%	83.80%	88.48%

multi-precision CNN, has the potential to achieve higher accuracy with fewer parameters than the model-wise quantization like BCNN. Therefore, to improve the neural network accuracy after quantization and to apply the RRAM accelerator to complicated application scenarios, it is important to take the accuracy loss caused by hardware computing deviations into consideration and enable the single bit RRAM to support multi-precision CNNs.

In this paper, we propose a configurable multi-precision CNN computing framework based on single bit RRAM, leveraging the software and hardware co-design. It consists of a layer-wise quantization algorithm considering hardware overhead and an architecture design supporting the multi-precision CNN computing. The main contributions of this paper include:

(1) **In the software level**, we propose an RRAM computing overhead aware neural network quantization and compression algorithm. It performs the layer-wise quantization based on the RRAM computing deviation, storage consumption, and latency. As a result, it is able to achieve equivalent accuracy as full precision computing with lower storage burden and latency.

(2) **In the hardware level**, we design a configurable multi-precision CNN computing architecture based on single bit RRAM, which implements data splicing in a flexible way and supports multi-precision computing. Besides, on top of the computing characteristics of the architecture, we optimize the mapping strategy for different network models to further improve the performance.

(3) Based on the proposed quantization method and the multi-precision computing architecture, we explore and analyze the trade-off between accuracy and hardware performance (latency, area, and energy consumption) under different hardware configurations and quantization settings. We also propose an optimal hardware configuration when mapping multi-precision CNN models.

(4) Experiment results show that, 70% computing area and 75% computing energy reduction can be achieved by the proposed framework. The equivalent energy efficiency is 3.44TOPs/W, which is 1.6 ~ 8.6× compared with existing RRAM based architectures.

2 PRELIMINARY

2.1 Convolutional Neural Network

Classical CNNs are usually composed of several convolutional (CONV) layers and fully-connected (FC) layers. CONV layers execute the convolution operation which can be expressed as:

$$f_o(x, y, z) = \sum_{i=0}^{K-1} \sum_{j=0}^{K-1} \sum_{k=1}^{C_{in}} f_i(x+i, y+j, k) k_z(i, j, k) \quad (1)$$

where 3-dimensional matrices f_o and f_i represent the input and output feature maps, respectively; k_z represents the z^{th} 3-dimensional convolution kernel with size of $K \times K \times C_{in}$; K is the kernel size and C_{in} is the number of input channels. The number of kernels is equal to the output channel number as denoted as C_{out} . And FC layers are similar to CONV layers.

2.2 RRAM Basics

RRAM is a candidate of emerging non-volatile memory, which has advantages of high density, low latency and energy, and high on/off ratio [22]. Multiple RRAM cells can be organized as a crossbar structure. By applying the input voltage vector $\mathbf{V}$ to the word line (WL) of a crossbar, we can get the output current vector $\mathbf{I}$ from each bit line (BL). The relationship between the input and the output can be expressed as [11]:

$$i_{out,k} = \sum_{j=1}^N g_{k,j} v_{in,j} \quad (2)$$

where $v_{in,j}$ represents the j -th element of the input voltage vector $\mathbf{V}$; $g_{k,j}$ represents the conductance of RRAM device in the k -th column and the j -th row; $i_{out,k}$ is the k -th element of the output current vector $\mathbf{I}$. Besides, since RRAM computes in the analog domain (i.e., currents and conductances) and the analog output signals are difficult to be stored, there are some necessary interfaces between crossbars and peripheral digital circuits, such as Analog-to-Digital Converters (ADCs), Digital-to-Analog Converters (DACs), and Sensing Amplifiers (SAs).

3 RRAM-AWARE QUANTIZATION

As mentioned in Section 1, single bit RRAM devices have higher reliability and perform computing more accurately than the multi-bit RRAM devices, but bring larger area overhead and heavier storage burden. One effective solution of the above problem is data quantization. As shown in Figure 1, different layers in one NN model have different quantization sensitivities. Thus, layer-wise quantization and compression are helpful to maintain the accuracy while further pushing the compression rate higher.

3.1 RRAM Computing Deviation

When mapping the multi-precision CNN model to RRAM based computing system, the accuracy loss not only comes from the compression error in the software level, but also depends on the hardware intrinsic computing error.

To explore the impacts of the RRAM computing deviations, we map a well-trained multi-precision CNN model, which is trained on GPU platform with traditional quantization method, onto the RRAM based computing system for testing. The results are shown in Table 1, where the baseline is floating-point computing on GPU, and the GPU quantization and GPU quantization on RRAM represent mapping the well-trained model to GPU and RRAM platforms, respectively. The results demonstrate that the multi-precision model, which is even well trained on GPU, still suffers from the high accuracy loss on RRAM computing systems. Thus, we have to analyze the deviation of RRAM computing systems and combine the RRAM computing deviation with the quantization method.

Since the R-ratio of RRAM is high and the variations impact of single bit RRAM is much lower, the device-level and circuit-level non-ideal factors, e.g., resistance variations will not affect the accuracy [17]. The mainly computing deviation and error in RRAM based computing system comes from the quantization error of the analog/digital interfaces. Assuming that the input data of crossbars is m -bit, the crossbar size is N , and the precision of SAs and ADCs is Q -bit. Ideally, the bit line current precision should be:

$$Q_{ideal} = m + \log_2(N) + 1 \quad (3)$$

The difference between Q and Q_{ideal} can cause significant quantization error. Based on this, we model the deviation and error of the RRAM based computing system and setup an RRAM computing behavior level simulator.

3.2 Optimization Model of RRAM-Aware Quantization

The overall goal of the quantization method is to determine the quantization precision of each layer to meet the requirements for the hardware performance. On top of the computing accuracy, we focus on the storage usage and latency for establishing the optimization model, since these two are directly related to the quantization precision of weights and activations.

Storage. Storage is reflected in the number of crossbars to store the weights. Assuming the layer number is L , the weights of layer i is W_i -bit, the crossbar size is N . $2 \times W_i$ crossbars are needed for storing one weight value (considering the signed data). Then the total number of crossbars is:

$$Num_c = \sum_{i=1}^L 2 \times W_i \times \left\lceil \frac{K_i^2 C_{in_i}}{N} \right\rceil \times \left\lceil \frac{C_{out_i}}{N} \right\rceil \quad (4)$$

Latency. Since the frequency of the digital modules is usually much higher than that of RRAM crossbars, we only consider the RRAM crossbar computation time as the main part of latency. The latency is decided by two factors: activation precision and the sliding times of the sliding window. Because the precision of DACs is limited due to the area and power limitations, the activation needs to be split and loaded to the word lines in multiple cycles. Assume the activation precision is M_i -bit, the precision of DACs is m -bit, sliding times of layer i is S_{t_i} , the total latency of crossbars is:

$$Latency = \sum_{i=1}^L \frac{M_i}{m} \times S_{t_i} \quad (5)$$

Num_c and $Latency$ are two proxies for estimating the RRAM computing overhead roughly but conveniently. The optimization model of RRAM computing overhead in quantization is shown in Equation (6), whose optimization target is the weighed sum of storage and latency.

$$\min_{\{W_i\}\{M_i\}} (\alpha Num_c(\{W_i\}, \{M_i\}) + \beta Latency(\{W_i\}, \{M_i\})) \quad (6)$$

s.t. $Loss(\{W_i\}, \{M_i\}) \leq Loss_Threshold$

3.3 Details of the Quantization Scheme

Based on the optimization model in Section 3.2, we propose the RRAM Computing Deviation Aware Quantization Scheme, which is shown in Algorithm 1. First, we set an accuracy loss threshold. Then, we try to solve Equation 6 for each layer. For the weights, reducing the required storage space is more essential, so we enlarge the quantization ratio of the layer with more weights. Therefore, we modify the initial accuracy loss threshold as shown in Line 6, where γ is a given empirical value (set to 2 in the experiments). After selecting the weight precision of each layer (Line 8 to Line 13), we retrain the entire CNN model. For the activation which has decisive impact on the computation latency, we focus more on the layer with a larger sliding times.

Algorithm 1 RRAM Computing Deviation Aware Quantization and Compression Scheme

Input: The CNN model, RRAM accelerator config, Accuracy loss threshold $Thres$

Output: Weights and activation precisions of each layer; Accuracy loss of the quantized CNN model.

```

1: Calculate the weights number of the entire CNN:  $W$ ;
2: Calculate the total sliding times:  $St$ ;
3: function CalculatePrecision ( $a_i, A, \gamma$ ) {
4:   for Each layer  $l_i$  do
5:     Calculate the proportion of  $l_i$ :  $p_i\% = a_i/A$ ;
6:     Update the loss threshold of  $l_i$ :  $Thres_i = Thres(1 + \gamma \cdot p_i\%)$ ;
7:     Define loss difference:  $\epsilon = 1$ ;
8:     for Different weights precision  $j$  do
9:       Simulate the accuracy loss  $loss_{(i,j)}$ ;
10:      if  $|loss_{(i,j)} - Thres_i| < \epsilon$  then
11:         $\epsilon = |loss_{(i,j)} - Thres_i|$ ; Select the precision  $j$ ;
12:      end if
13:    end for
14:  end for
15: return  $\{l_i, j\}$  }
16: // For weights:  $\{l_i, j\} = CalculatePrecision(w_i, W, \alpha)$ ;
17: Retrain CNN with selected weights precision of each layer;
18: // For activation:  $\{l_i, j\} = CalculatePrecision(s_i, St, \beta)$ ;
19: Retrain CNN with selected activation precision of each layer;
20: return Weight precision, Activation precision, and Accuracy loss;

```

4 ARCHITECTURE DESIGN

As mentioned in Section 3, our multi-precision quantization approach can improve the classification accuracy, and reduce the storage burden and computing latency further. To support the acceleration of multi-precision CNNs on RRAM computing platforms, we propose a configurable multi-precision CNN computing architecture based on single bit RRAM. The primary computing units of the architecture are composed of single bit RRAMs, which have more mature fabrications to perform more reliable computations.

4.1 Storing Multi-bit Weights in RRAM

In the existing work, to represent a higher bit-width value with low bit RRAM cells, ISAAC [15] and PRIME [3] use multiple cells in the same row to store one weight value and merge the results of adjacent columns to get the final results. The peripheral circuits connected to each bit line should be customized to support shift-and-add operation, which not only increases the overhead of the peripheral circuits of crossbars but also lacks in reconfigurability for supporting MVMs with different bit widths.

In our architecture design, we use multiple crossbars to store multi-bit weights. Different numbers of computing units of the same design are used for different precision weights. Therefore, we do not need to reconfigure the peripheral circuits of crossbars (e.g. SAs and ADCs). Our design improves the flexibility of computing units while reducing the overhead of peripheral circuits.

4.2 Data Splitting Implementation

Supposing the bit width of the weights is W -bit, the input feature data is M -bit, and the precision of DACs is m -bit, then Equation (1)

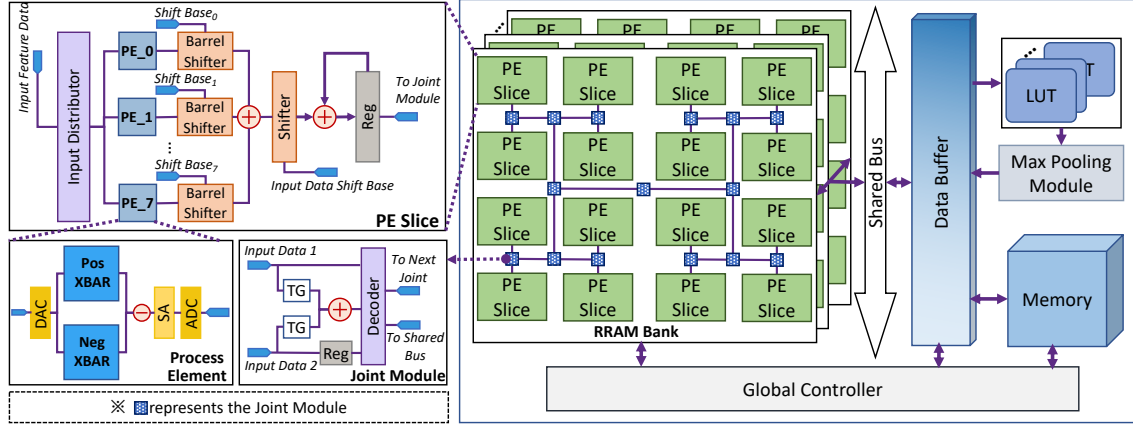


Figure 2: Right: The overall architecture design; Left: Details of the Process Element (PE), the PE Slice, and the Joint Model

can be rewrite as Equation (7):

$$f_o(x, y, z) = \sum_{i=0}^{K-1} \sum_{j=0}^{K-1} \sum_{k=1}^{C_{in}} f_i(x+i, y+j, k) k_z(i, j, k)$$

$$= \sum_{c=0}^{M/m-1} \left[2^{mc} \sum_{w=0}^{W-1} 2^w \left(\sum_{i=0}^{K-1} \sum_{j=0}^{K-1} \sum_{k=1}^{C_{in}} f_{ic}(x+i, y+j, k) k_{zw}(i, j, k) \right) \right] \quad (7)$$

where k_{zw} is the w -th bit of k_z , f_{ic} are the $(mc \sim m(c+1))$ -th bits of f_i . From Equation (7), the inner multiply-and-accumulate can be calculated by RRAM crossbars and the following multiplications can be achieved by left shift operations. Thus, the spatial parallel and temporal serial mapping strategies are used for weights and input feature data respectively.

For the weights mapping. Our architecture stores the multi-precision data in the same position of different crossbars, instead in consecutive positions in the same row of the same devices, as demonstrated in Section 4.1.

For the input feature data mapping. Constrained by the area limitation, we connect one low resolution DAC to each Word Line, e.g., 2-bit DACs. So, when the precision of input feature data is larger than two, multiple cycles are needed to finish the computation.

4.3 Architecture Details

The overall architecture details are shown in Figure 2. From the lower level to the higher level, the architecture is composed of Process Elements (PE), PE Slice, RRAM Bank, and some other processing and storage units. RRAM Banks and other processing and storage units are connected with a Shared Bus. The data flow and instruction executions are controlled by a Global Controller.

Process Element. PE mainly consists of two crossbars and their peripheral circuits. DACs first convert the input feature data into voltages with different levels and feed them into the two crossbars in parallel. Since the weights are signed value while the conductance of RRAM is positive, we use two sets of crossbars, i.e., positive crossbars (Pos XBAR) and negative crossbars (Neg XBAR), to store the weights value except for the sign bit. After the MVMs are finished by crossbars, the currents of the two corresponding BLs are subtracted by an analog subtraction unit. The subtraction can also offset the High-Resistance-State leakage current to deliver results of high accuracy. Then, the merged analog current is amplified and converted to the digital signals by SAs and ADCs. It is worth

mentioning that due to the large area and high energy consumption of ADCs, multiple BLs share one SA and DAC to reduce the energy costs and avoid the footprint mismatch problem.

PE Slice. Each PE Slice consists of an Input Distributor, eight PEs and Barrel Shifters, and a Shift-And-Add Unit. The Input Distributor fetches the input feature data from the Shared Bus, splits the data into multiple m -bit data, and loads them to eight PEs in each cycle. The eight PEs in one PE Slice store at most eight bits of one weight value; existing work has demonstrated 8-bit weights will not cause accuracy loss [20]. As shown in Equation (7), the results of each PE should be multiplied with 2^w , which is equivalent to shift w -bit to the left. The Barrel Shifter connected to the PE performs the left shift operation with various shift base. After summing the shifted values, the second shifter performs the secondary multiplication with 2^{mc} in Equation (7). Since the input feature data is split and loaded in multiple cycles, a register (Reg) and an accumulator are used for calculating the final results.

PE Bank. When the length of the unrolled weight is larger than the crossbar size, the column vector of inputs and weights are split for mapping onto different crossbars [16]. In our architecture design, since one PE Slice is equivalent to a crossbar based on 8-bit RRAM cells, we need to utilize the results of multiple PE Slices for mapping the long kernel. Thus, adequate PE Slices are placed in one RRAM Bank (i.e., 16 256×256 crossbars), to reduce the costs of data movements between different RRAM Banks. Moreover, in order to reduce the costs of data transferring further, the PE Slices in one RRAM Bank are linked as an H-Tree structure, as shown in Figure 2. Each connection node of the H-Tree is a **Joint Module**, which manages the data forwarding and summations of PE Slice results. Before the summation of PE Slice results is finished, the Joint Module calculates the addition of two PE Slice results and output the results to the following Joint Module. Otherwise, the Joint Module forward the sum or original input data to the Shared Bus for storing the result in Data Buffer.

Other Storage and Processing Units. When the computations of the CONV/FC layer are finished, the output results must be stored for subsequent operations, such as Pooling and ReLU. In our design, we use 512×512 RRAM crossbars [19] as Data Buffer, Look-Up-Table (LUT) for BatchNormal (BN) and ReLU, and Max Pooling Module for Pooling.

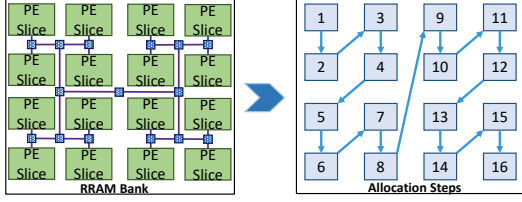


Figure 3: The schematic diagram of the allocation strategy, the number represents the order of allocations

4.4 Mapping Strategy

For the purpose of mapping a CNN model onto the proposed architecture, the mapping strategy mainly includes two steps:

(1) Determine the number of PE Slices and RRAM Banks for each layer: supposing the number of PE Slices in one RRAM Banks is 2^S . Then we can calculate the number of PE Slices for each kernel and the number of RRAM Banks of the whole layer in Equation (8) (2^S is set larger than N_{PES}):

$$N_{PES} = \left\lceil \left\lfloor \frac{K^2 \times C_{in}}{N} \right\rfloor \times W/8 \right\rceil; N_{Bank} = \left\lceil \frac{[C_{out}/N]}{\lfloor 2^S/N_{PES} \rfloor} \right\rceil \quad (8)$$

(2) Optimize the PE Slice allocation of each layer: since RRAM Banks use the H-Tree structure, the delay and energy consumption caused by summing the two PE Slice results will depend on the position of the PE Slices. The optimized allocation strategy is described as a diffusion process, which spreads along the diagonal, as shown in Figure 3.

5 HARDWARE CONFIGURATION ANALYSIS

In the proposed architecture, different configurations of the PE will directly influence the computing accuracy and the hardware overhead. The main design parameters of PEs include the design choices of the interfaces and the crossbar size. This section will introduce the trade-off between accuracy and hardware performance considering different design parameters.

Impact of DACs. The precision of DACs determines the maximum bit width of the input voltages of crossbars. On one hand, when DACs precision is lower than the activation precision, multiple cycles are needed for loading data, which increases the latency. However, most of the activation precision is reduced by 50% using our proposed quantization method, high precision DACs are wasteful. On the other hand, Saberi et al. [14] demonstrated that the area and power of DACs grow exponentially with the increasing precision; therefore, a high precision DAC must be shared by multiple WLS, and we still need multiple cycles for loading activation to different WLS.

Impact of crossbar size. According to Equation (3), the crossbar size influences the ideal precision of the BL current, and affects the RRAM computing error further. Besides, since the crossbar size determines the number of crossbars we need for storing the weights, it will also affect the hardware area and energy overhead [24]. Combining the above considerations, theoretically, crossbars with the larger size have the advantage of lower energy consumption, but the area and the computing accuracy is slightly worse compared with smaller crossbars.

Impact of SAs and ADCs. Firstly, the precision of the analog-to-digital interfaces affects the computing accuracy as demonstrated

Table 2: Accuracy and Hardware Overhead of Different System Configurations. (Q is the precision Q_{ADC} of the analog-to-digital interfaces; DACs are set to 2-bit, the units of area, energy, and latency are mm^2 , mJ , and ms , respectively)

LeNet								
Crossbar size = 128			Crossbar size = 256			Crossbar size = 512		
	Q=6	Q=8	Q=10	Q=6	Q=8	Q=10	Q=6	Q=10
Accuracy	76.00%	76.64%	76.44%	75.96%	76.52%	76.54%	75.36%	76.64%
Area	5.45	5.54	5.56	13.98	14.11	14.13	45.7	45.92
Energy	0.005	0.006	0.015	0.004	0.006	0.011	0.008	0.01
Latency	0.91	1.82	1.82	0.91	1.82	1.82	0.91	1.82
VGG-16								
Crossbar size = 128			Crossbar size = 256			Crossbar size = 512		
	Q=6	Q=8	Q=10	Q=6	Q=8	Q=10	Q=6	Q=10
Accuracy	91.86%	92.04%	92.36%	91.80%	92.20%	92.32%	92.12%	91.70%
Area	296.45	301.85	302.59	319.21	322.23	322.64	455.14	468.39
Energy	0.402	0.613	1.793	0.239	0.348	0.943	0.162	0.22
Latency	2.88	5.76	5.76	2.88	5.76	5.76	2.88	5.76
ResNet-18								
Crossbar size = 128			Crossbar size = 256			Crossbar size = 512		
	Q=6	Q=8	Q=10	Q=6	Q=8	Q=10	Q=6	Q=10
Accuracy	94.62%	94.90%	95.04%	94.78%	94.94%	95.06%	94.42%	94.94%
Area	422.29	429.98	431.04	454.35	458.65	459.24	676.36	679.62
Energy	2.317	3.539	10.404	1.352	1.973	5.421	0.893	1.219
Latency	6.96	13.93	13.93	6.96	13.92	13.93	6.96	13.93

in Section 3.1. However, with the improvement of the interfaces precision, the area and power of interfaces will increase dramatically. Furthermore, due to the area limitations, multiple BLs share one SA and ADC. If the interface area becomes larger, the parallelism of the output will degrade, causing longer computing latency.

More quantitative details are shown in Section 6.2.

6 EXPERIMENTAL RESULTS

6.1 Experiment Setup

We use three types of CNN models as our benchmarks: LeNet [10], modified VGG-16 [8], and ResNet-18 [5]. All experiments are evaluated on the Cifar-10 dataset [7]. We synthesize the digital circuit modules at 45nm technology node with 500MHz using Cadence Encounter[®] RTL Compiler. We model the energy and area consumption for RRAM devices at 100MHz using the results from [1], for ADCs using the results from [3, 4, 9], and for 1-bit and 2-bit DACs using the results in [14].

6.2 Trade-off between Accuracy and Hardware Overhead

Table 2 shows the CNN accuracy and hardware overhead in PE (*i.e.*, RRAM Bank) under different hardware configurations. It demonstrates that as the crossbar size increases, the average energy decreases in general, but both the accuracy loss and the area become larger. Meanwhile, for the digital-to-analog interface, higher precision interface brings higher CNN accuracy because of the low quantization error, but it also brings larger energy and latency. These trends accord with our theoretical analysis in Section 5.

We can also observe that 512×512 crossbars take up $1.5\times$ area cost than 256×256 crossbars but do not gain much energy efficiency w.r.t. the small CNN like LeNet, while 256×256 crossbars only have 8% area overhead compared with 128×128 crossbars but reduce the energy consumption by $1.2\times$ to more than $1.8\times$. In the meantime, 6-bit digital-to-analog interface has the least latency, energy consumption and area cost compared to interfaces with other precision. Table 3 shows the impact of DAC precision on both CNN accuracy and accelerator performance using crossbar size of 256 and ADC precision of 6-bit. Apparently,

Table 3: Accuracy and Hardware Overhead under Different DAC precision Q_{DAC} . (crossbar size and output interface precision are set to 256 and 6-bit)

	Accuracy	Area/ mm^2	Energy/ mJ	Latency/ ms
1-bit	91.92%	319.16	0.465	5.76
2-bit	91.80%	319.21	0.239	2.88

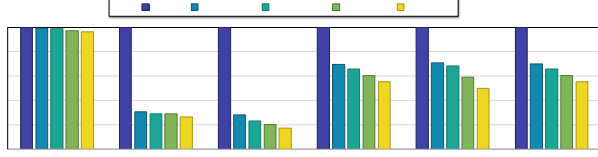


Figure 4: VGG16 Accuracy on Cifar10 dataset and Hardware Performance under Different Accuracy Loss Thresholds

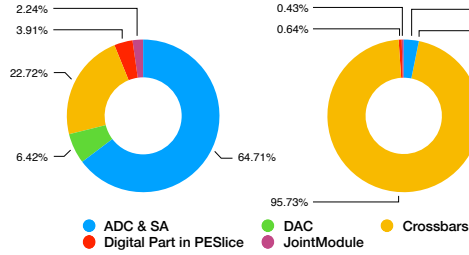


Figure 5: The Energy (left) and Area (right) Breakdown

2-bit DACs can achieve lower energy consumption and latency with almost the same accuracy and area. Therefore, we choose the $\{Crossbar\ size, Q_{ADC}, Q_{DAC}\} = \{256, 6, 2\}$ as our hardware configuration for the following experiments. Figure 4 shows the classification accuracy of VGG16 on Cifar10 and hardware performance after quantization under different accuracy loss threshold settings. From the results, we see that the accuracy loss increases slightly, as the quantization ratio increased. Because the precision of weights decreased, the area for storing weights becomes smaller, which occupies the main part of the RRAM Bank. Besides, the lower activation precision is benefit for reducing the latency, as demonstrated in Section 3.2. Since the total amount of computing data decreases, the energy of computing units and data movements and the data movement latency are reduced. We choose 0.005 as our loss threshold to get better performance with nearly no accuracy loss for the following experiments.

6.3 Performance Analysis

With above hardware configurations and accuracy loss threshold setting, the equivalent energy efficiency of the computing units (*i.e.*, RRAM Banks) is $3.44Tops/W$, nearly $8.6\times$ and $1.6\times$ compared with existing RRAM-based accelerators, ISAAC [15] and PRIME [3], respectively. Compared with these architectures, the extra overhead is introduced by the additional digital circuits in the PE Slice and the Joint Module. From Figure 5, we see that the extra energy and area overhead cover only 6% and 1.07% w.r.t. the RRAM banks.

7 CONCLUSION

In this paper, we propose a configurable multi-precision CNN computing framework based on single bit RRAM, which consists of an RRAM computing overhead aware network quantization algorithm and a configurable multi-precision CNN computing architecture based on single bit RRAM. The proposed method can achieve

equivalent accuracy as full precision CNN but with lower storage consumption and latency via multiple precision quantization. The designed architecture supports for accelerating the multi-precision CNNs even with various precision among different layers. An optimal hardware is also proposed after analyzing the trade-off between accuracy and hardware performance. Experiment results demonstrate that our proposed framework can reduce 70% computing area and 75% computing energy on average with little accuracy loss, and the equivalent energy efficiency is $1.6 \sim 8.6\times$ compared with existing RRAM based architectures.

8 ACKNOWLEDGEMENTS

This work was supported by National Key Research and Development Program of China (No. 2017YFA0207600), National Natural Science Foundation of China (No. 61832007, 61622403, 61621091), Beijing National Research Center for Information Science and Technology.

REFERENCES

- [1] M. Chang et al. 2014. 19.4 embedded 1Mb ReRAM in 28nm CMOS with 0.27-to-1V read using swing-sample-and-couple sense amplifier and self-boost-write-termination scheme. In *ISSCC, 2014*. 332–333.
- [2] W. Chen et al. 2018. A 65nm 1Mb nonvolatile computing-in-memory ReRAM macro with sub-16ns multiply-and-accumulate for binary DNN AI edge processors. In *ISSCC, 2018*. 494–496.
- [3] P. Chi et al. 2016. PRIME: A Novel Processing-in-memory Architecture for Neural Network Computation in ReRAM-based Main Memory. In *ISCA, 2016*.
- [4] K. D. Choo et al. 2016. 27.3 Area-efficient 1GS/s 6b SAR ADC with charge-injection-cell-based DAC. In *ISSCC, 2016*. 460–461.
- [5] K. He et al. 2016. Deep Residual Learning for Image Recognition. In *CVPR, 2016*.
- [6] M. Hu et al. 2016. Dot-product engine for neuromorphic computing: Programming 1T1M crossbar to accelerate matrix-vector multiplication. In *DAC, 2016*.
- [7] Kaggle et al. 2014. CIFAR-10 - Object Recognition in Images. website. (2014). <https://www.kaggle.com/c/cifar-10>.
- [8] S. Karen et al. 2014. Very Deep Convolutional Networks for Large-Scale Image Recognition. *Computer Science* (2014).
- [9] L. Kull et al. 2017. 28.5 A 10b 1.5GS/s pipelined-SAR ADC with background second-stage common-mode regulation and offset calibration in 14nm CMOS FinFET. In *ISSCC, 2017*. 474–475.
- [10] Y. LeCun et al. 1998. Gradient-based learning applied to document recognition. In *Proceedings of the IEEE, 1998*. 2278–2324.
- [11] B. Li et al. 2015. MErging the Interface: Power, area and accuracy co-optimization for RRAM crossbar-based mixed-signal computing system. In *DAC, 2015*.
- [12] J. Lin et al. 2018. Rescuing memristor-based computing with non-linear resistance levels. In *DATE, 2018*. 407–412.
- [13] M. Rastegari et al. 2016. Xnor-net: Imagenet classification using binary convolutional neural networks. In *ECCV, 2016*. Springer, 525–542.
- [14] M. Saberi et al. 2011. Analysis of Power Consumption and Linearity in Capacitive Digital-to-Analog Converters Used in Successive Approximation ADCs. *IEEE Transactions on Circuits and Systems I: Regular Papers* 58, 8 (Aug 2011), 1736–1748.
- [15] A. Shafiee et al. 2016. ISAAC: A Convolutional Neural Network Accelerator with In-situ Analog Arithmetic in Crossbars. In *ISCA, 2016*.
- [16] L. Song et al. 2017. PipeLayer: A Pipelined ReRAM-Based Accelerator for Deep Learning. In *HPCA, 2017*.
- [17] X. Sun et al. 2018. XNOR-RRAM: A scalable and parallel resistive synaptic architecture for binary neural networks. In *DATE, 2018*. 1423–1428.
- [18] T. Tang et al. 2017. Binary convolutional neural network on RRAM. In *ASPDAC, 2017*. 782–787.
- [19] H. S. P. Wong et al. 2012. Metal Oxide RRAM. *Proc. IEEE* 100, 6, 1951–1970. <https://doi.org/10.1109/JPROC.2012.2190369>
- [20] S. Wu et al. 2018. Training and inference with integers in deep neural networks. *arXiv preprint arXiv:1802.04680* (2018).
- [21] L. Xia et al. 2016. Switched by input: Power efficient structure for RRAM-based convolutional neural network. In *DAC, 2016*. 1–6.
- [22] S. Yu and P. Chen. 2016. Emerging Memory Technologies: Recent Trends and Prospects. *IEEE Solid-State Circuits Magazine* 8, 2 (Spring 2016), 43–56.
- [23] S. Zhou et al. 2016. Dorefa-net: Training low bitwidth convolutional neural networks with low bitwidth gradients. *arXiv preprint arXiv:1606.06160* (2016).
- [24] Z. Zhu et al. 2018. Mixed Size Crossbar based RRAM CNN Accelerator with Overlapped Mapping Method. In *ICCAD, 2018*. 1–6.